Tema 5 – Gestión de Entradas y Salidas

Arquitectura software del sistema de E/S

Buffering

Planificación de disco

Sistemas RAID

Sistemas Operativos
Alejandro J. León Salas, 2021
Lenguajes y Sistemas Informáticos

Objetivos

- Comprender la arquitectura software de E/S y su estructura en capas para la gestión de los dispositivos.
- Conocer el funcionamiento básico de un controlador hardware de dispositivo (*device controller*) y su controlador software (*device driver*) asociado.
- Distinguir distintos esquemas de comunicación entre el procesador y los dispositivos
- Comprender los aspectos de diseño del SO respecto a los dispositivos de E/S para abordar los problemas que estos plantean.
- Conocer las distintas estrategias de *buffering* para mejorar el rendimiento.
- Conocer los componentes fundamentales de los discos magnéticos y comprender como influyen en el diseño de los planificadores de E/S de disco y los sistemas RAID.

Bibliografía

- [Sta2005] W. Stallings, Sistemas Operativos. Aspectos Internos y Principios de Diseño (5/e), Prentice Hall, 2005.
- [Sta2018] W. Stallings, Operating Systems. Internals and design principles (9th ed.). Pearson Education, 2018.

Contenidos

- Dispositivos de E/S
- Arquitectura software del sistema de E/S
- *Buffering*
- Planificación de disco
- Sistemas RAID

Dispositivos de E/S

- Técnicas de E/S
- Controlador hardware
- Requisitos del software de E/S

Técnicas de E/S

- **E/S programada**. El procesador envía una orden de E/S a un módulo de E/S y supervisa la terminación de la operación de E/S.
- **E/S guiada por interrupción**. El procesador envía una orden de E/S a un módulo de E/S y continua ejecutando instrucciones. El SO bloquea al proceso que solicita la orden. El procesador es interrumpido por el módulo de E/S al completar la orden y la RSI() correspondiente desbloquea al proceso.
- **Acceso directo de memoria** (*Direct Memory Access*, DMA). Un módulo de DMA controla el intercambio de datos entre la memoria principal y un módulo de E/S. El procesador envía una orden de transferencia de un bloque de datos al módulo de DMA y resulta interrumpido sólo cuando se haya transferido el bloque completo.

Técnicas de E/S

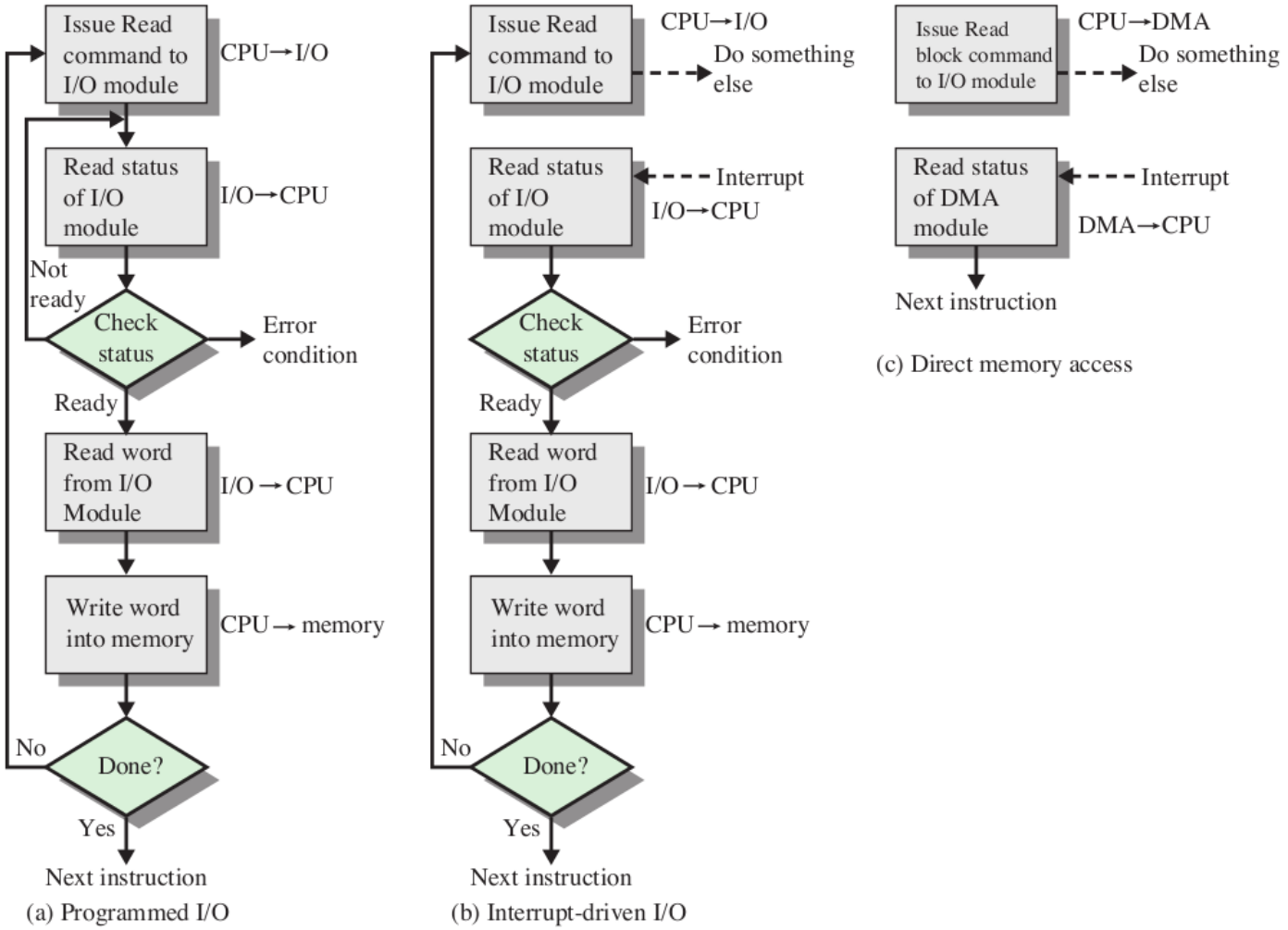


Figure 7.4 Three Techniques for Input of a Block of Data From [Sta2005]

Controlador hardware

- **Objetivo**:
  - Actuar de intermediario entre el procesador y el/los dispositivo(s) de E/S.
  - El SO se comunica exclusivamente con el controlador no con el/los dispositivo(s) que controla.
- **Estructura de un controlador genérico**. Se puede dividir en tres capas funcionales:
  - Interfaz con el bus del sistema.
  - Controlador (genérico) de dispositivo.
  - Interfaz con el/los dispositivo(s).

Controlador hardware

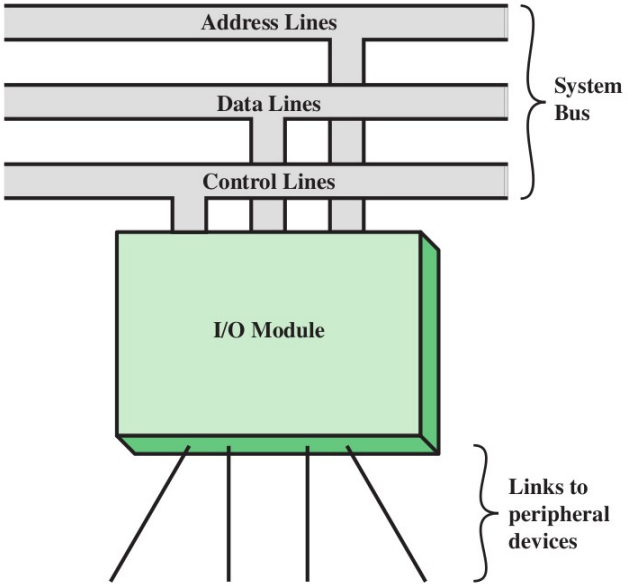


Figure 7.1 Generic Model of an I/O Module

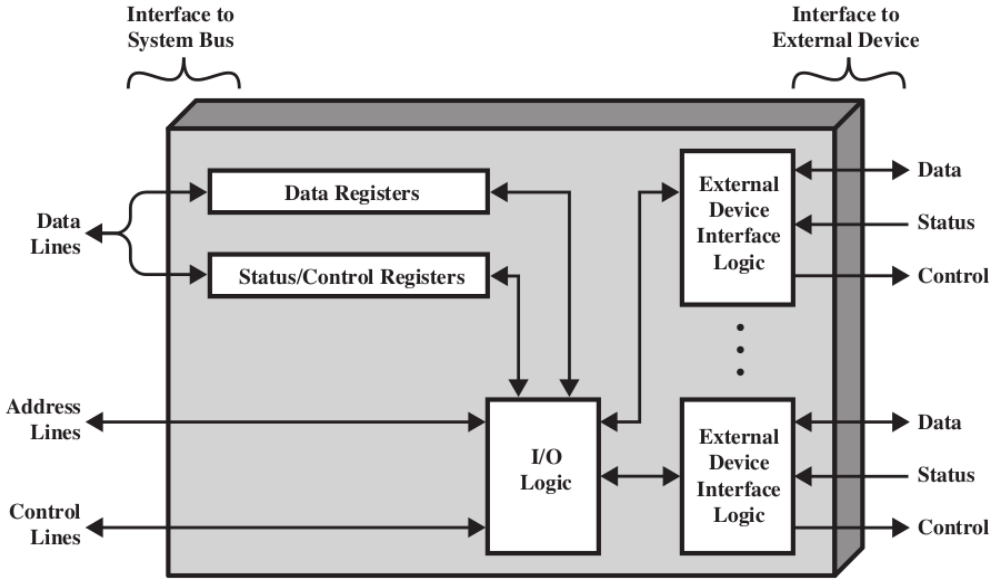
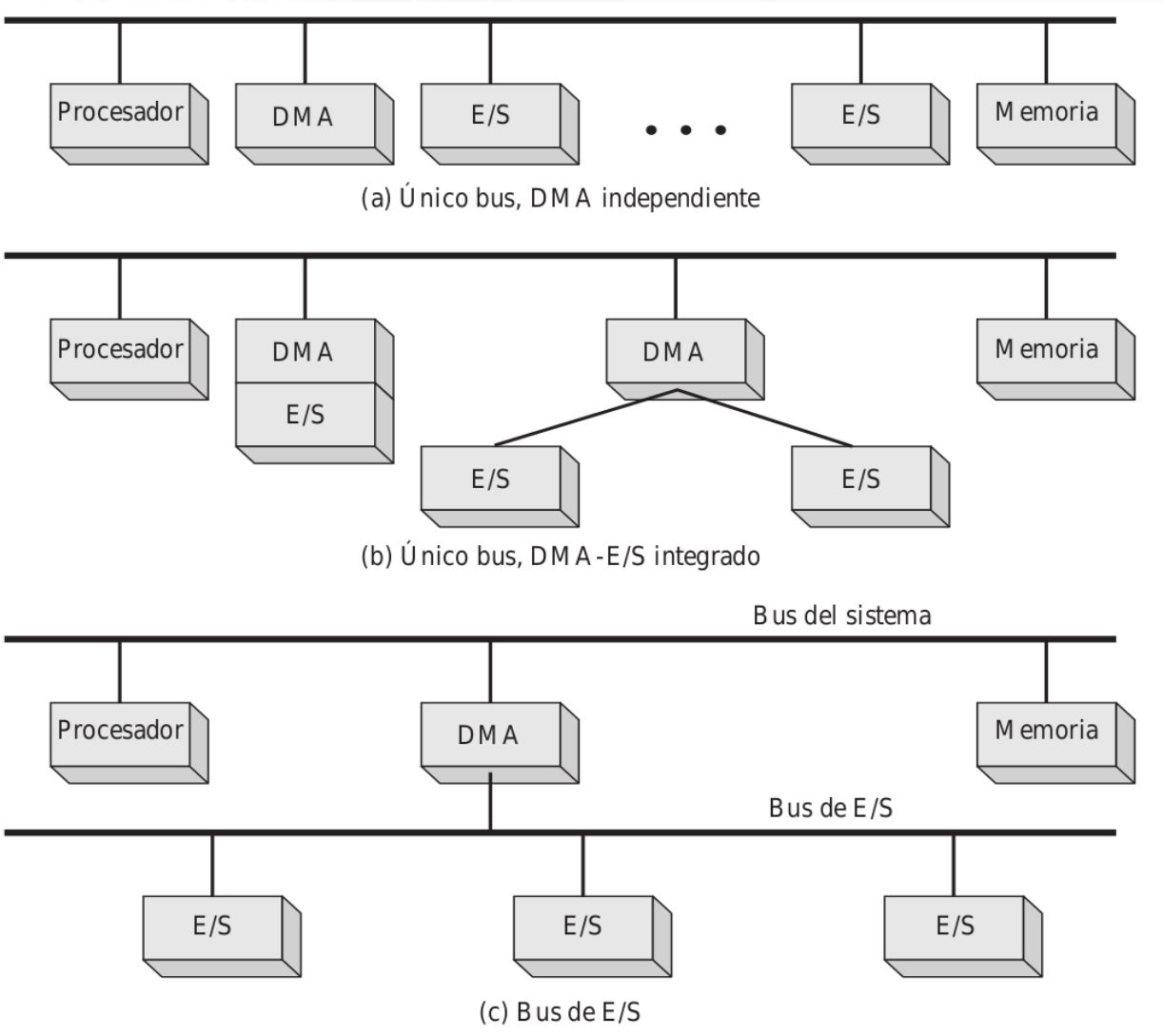


Figure 7.3 Block Diagram of an I/O Module From [Sta2005]

Controlador hardware



From [Sta2005]

Controlador hardware

Funciones:

- Control y temporización. Por ejemplo controlar la transferencia de datos al procesador.
- Comunicación con el procesador y con los dispositivos.
- Almacenamiento temporal de datos (*buffers*). La velocidad de transferencia entre los dispositivos y el módulo E/S es mucho menor que entre procesador/memoria, procesador/módulo o memoria/módulo.
- Detección de errores. Por ejemplo bit de paridad o códigos CRC en sectores de disco.

Requisitos del software de E/S

Funciones principales:

- Control de dispositivos.
- Ocultación de características y funcionamiento de los dispositivos al resto del SO.

Consideraciones:

- Funcionamiento asíncrono de CPU y dispositivos.
- Diferencia de velocidad entre CPU-RAM-dispositivo.
- Variaciones entre dispositivos:
  - Velocidad de transferencia.
  - Dispositivos orientados a bloques o a caracteres.
  - Dispositivos dedicados o compartidos.
  - Tipo de operación: `RD_ONLY`, `WR_ONLY`, `RDWR`

Dispositivos orientados a bloques/caracteres

- **Orientados a bloques**. (ej. Discos magnéticos)
  - Almacenan información en bloques de longitud fija.
  - Cada bloque tiene un identificador único que permite acceder al bloque deseado.
- **Orientados a caracteres**. (ej. Ratón)
  - Proporcionan un flujo de caracteres sin ninguna estructuración.
  - No se puede seleccionar un elemento concreto del flujo ya que no tienen identificadores para realizar búsquedas.
- El reloj del sistema es un dispositivo aparte que no se puede clasificar bajo esta categorización.

Contenidos

- Dispositivos de E/S
- Arquitectura software del sistema de E/S
- *Buffering*
- Planificación de disco
- Sistemas RAID

Arquitectura software del sistema de E/S

- Principios de diseño
- Arquitectura en capas
- Funcionamiento del software de E/S

Principios de diseño

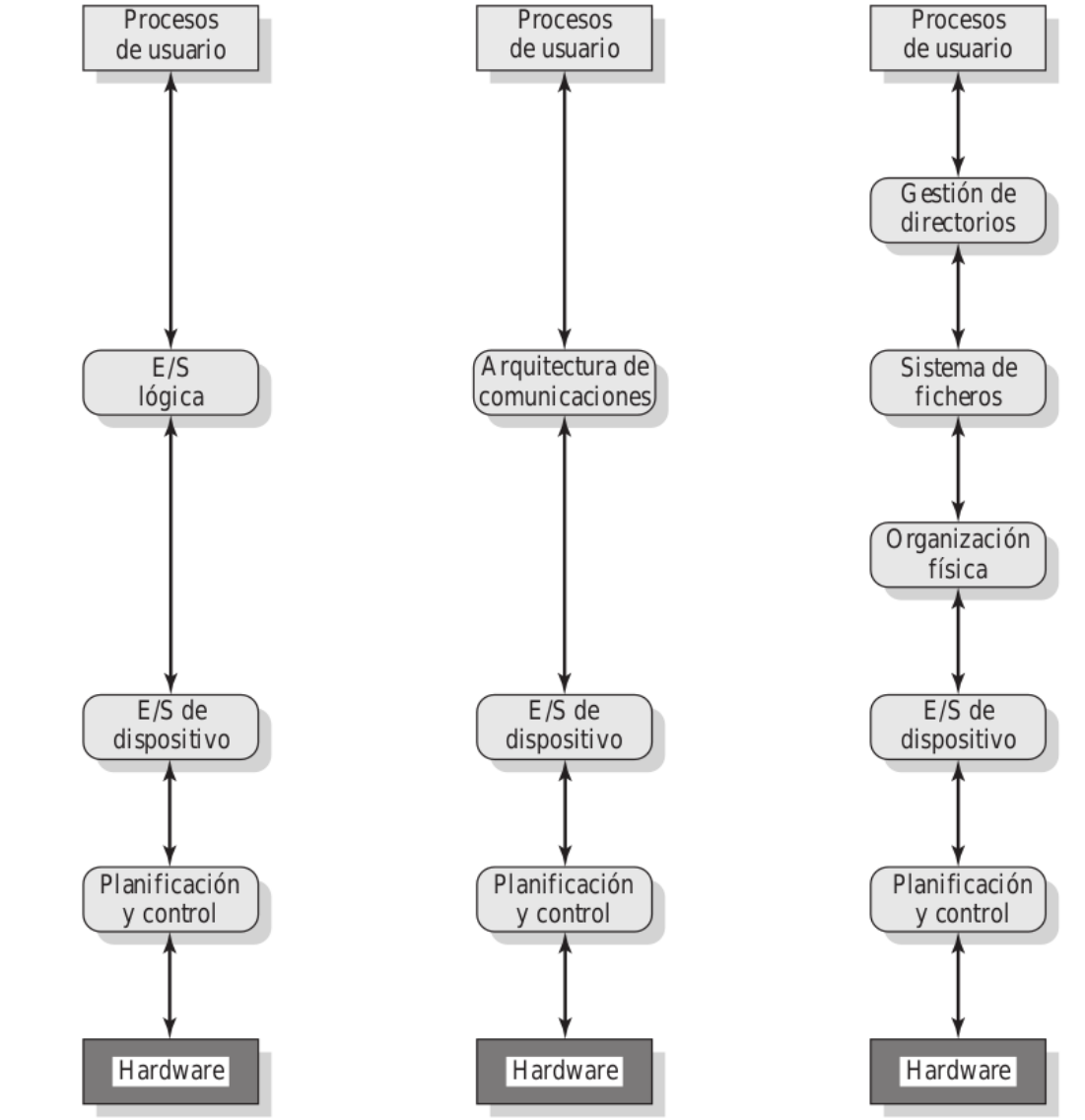
- **Eficiencia**:
  - Los dispositivos de E/S son lentos comparados con RAM y procesador.
  - Sol.1: Multiprogramación.
  - Sol.2: Intercambio a disco, ¡pero es una E/S!
  - Sol.3: Optimizar la E/S de disco.
- **Generalidad**:
  - Uniformidad de dispositivos desde el punto de vista de los procesos.
  - Uniformidad de gestión de dispositivos y operaciones de E/S por parte del *kernel*.

Principios de diseño

Por tanto:

- Estrategia de diseño modular y jerárquico para el sistema de E/S (**Arquitectura de capas**)
- Detalles del dispositivo de E/S en las capas de nivel inferior.
- Una capa proporciona servicios a la capa inmediatamente superior ocultando los detalles de implementación.
- Cambios en una capa no suponen cambios en capas de nivel superior (encapsulación).
- Tanto los procesos de usuario como el *kernel* utilizan los dispositivos en términos de funciones generales: abrir/cerrar, lectura/escritura.

Arquitectura en capas

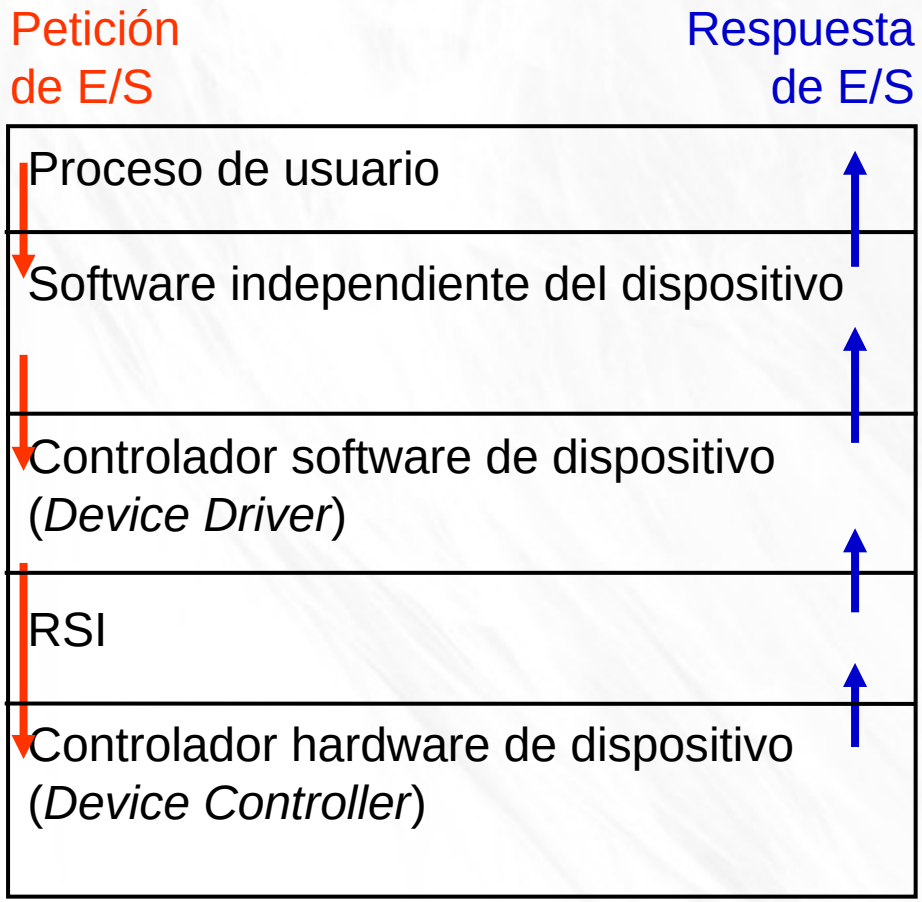


From [Sta2005]

Arquitectura en capas

- **E/S lógica**. La E/S lógica gestiona tareas genéricas de E/S para los procesos de usuario mediante un identificador de dispositivo y operaciones de alto nivel: abrir/cerrar, leer/escribir.
- **E/S de dispositivo**. La operaciones requeridas y los datos (caracteres en los *buffers*, registros, etc.) se convierten en las secuencias apropiadas de instrucciones de E/S para el controlador software. Se pueden utilizar técnicas de uso de *buffers* para mejorar la utilización.
- **Planificación y control**. Gestión y planificación de la cola de peticiones de E/S y control de las operaciones. En este nivel se manejan las interrupciones y se recoge el estado de la E/S y se informa del mismo. Aquí se encuentran el controlador software y la RSI.

Funcionamiento del software de E/S



Funciones de E/S:
Llamada de E/S

Asignación de nombres, protección, bloqueo/desbloqueo, buffering, asignación de dispositivos

Asignación de valores a registros de dispositivo, comprobación de estado

Informar de fin de E/S

Realización de la operación de E/S

Software en espacio de usuario

- **Biblioteca estándar de E/S**. Proporciona funciones de preparación y formato de E/S y en su implementación hacen uso de los *wrappers* de llamadas al sistema:

```
int printf(const char *format, ...);
```

- ***Wrappers*** de llamadas al sistema para E/S. Proporcionan servicios de E/S mediante la API del sistema operativo:

```
ssize_t write(int fd, const void *buf, size_t count);
```

- ***Spooling***. Un proceso de sistema que acepta solicitudes por parte de los procesos de usuario y realiza el trabajo requerido por éstas.

Software independiente de dispositivo

Funciones:

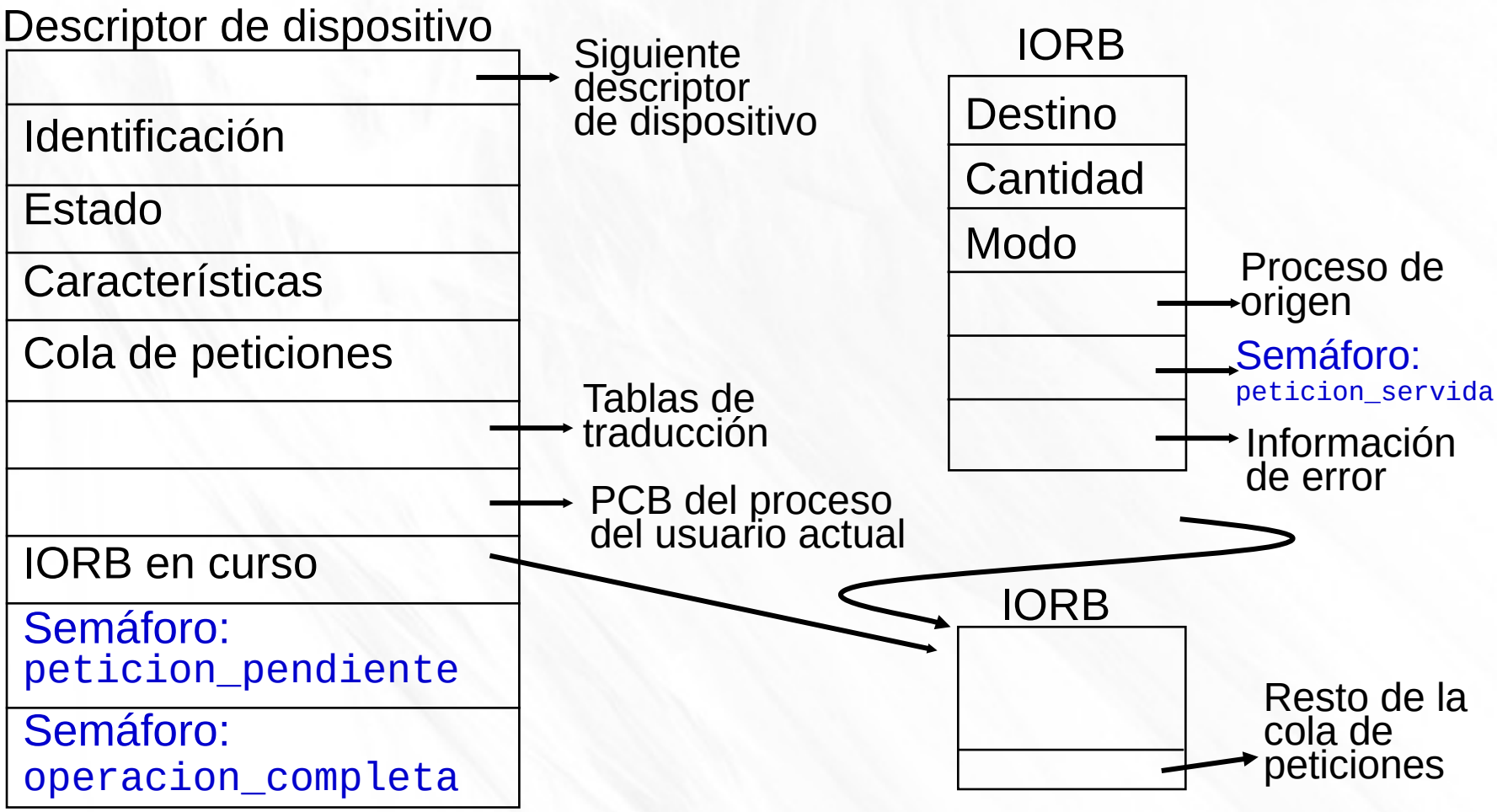
- Suministrar una interfaz uniforme al *kernel*.
- Asignar nombres a los dispositivos
- Proteger a los dispositivos de accesos no autorizados.
- Proporcionar un tamaño de bloque independiente de dispositivo (dispositivos de bloques).
- Proporcionar *buffering*.
- Asignar/liberar dispositivos dedicados.
- Gestión de errores.

Software independiente de dispositivo

Composición:

- Formado por **rutinas de E/S** a nivel *kernel* (`sys_write`) que son accedidas mediante llamadas al sistema, bien mediante *wrappers* bien directamente mediante código ensamblador y una trampa (`syscall`).
- Las rutinas de E/S tienen que ser código reentrante.
- **Estructuras de datos** asociadas a las rutinas de E/S:
- **Descriptor de dispositivo**. Mantiene la información del dispositivo asociado.
- Bloque de petición de E/S (**IORB**). Contiene la información correspondiente a una petición de E/S.

Software independiente de dispositivo



Software independiente de dispositivo

- Supongamos una petición de E/S desde un *wrapper* de llamada al sistema de E/S:

  ssize_t write(int fd, const void *buf, size_t count);

- Funcionamiento de una rutina de E/S genérica:
  - Identificar el dispositivo.
  - Comprobar los parámetros de la petición.
  - Empaquetar los parámetros en un IORB y enlazarlo en la cola del descriptor de dispositivo identificado.
  - Indicar al controlador software (*driver*) que se ha añadido una petición a su cola:

    `signal(peticion_pendiente)`

  - Esperar que el controlador software (*driver*) resuelva la petición:

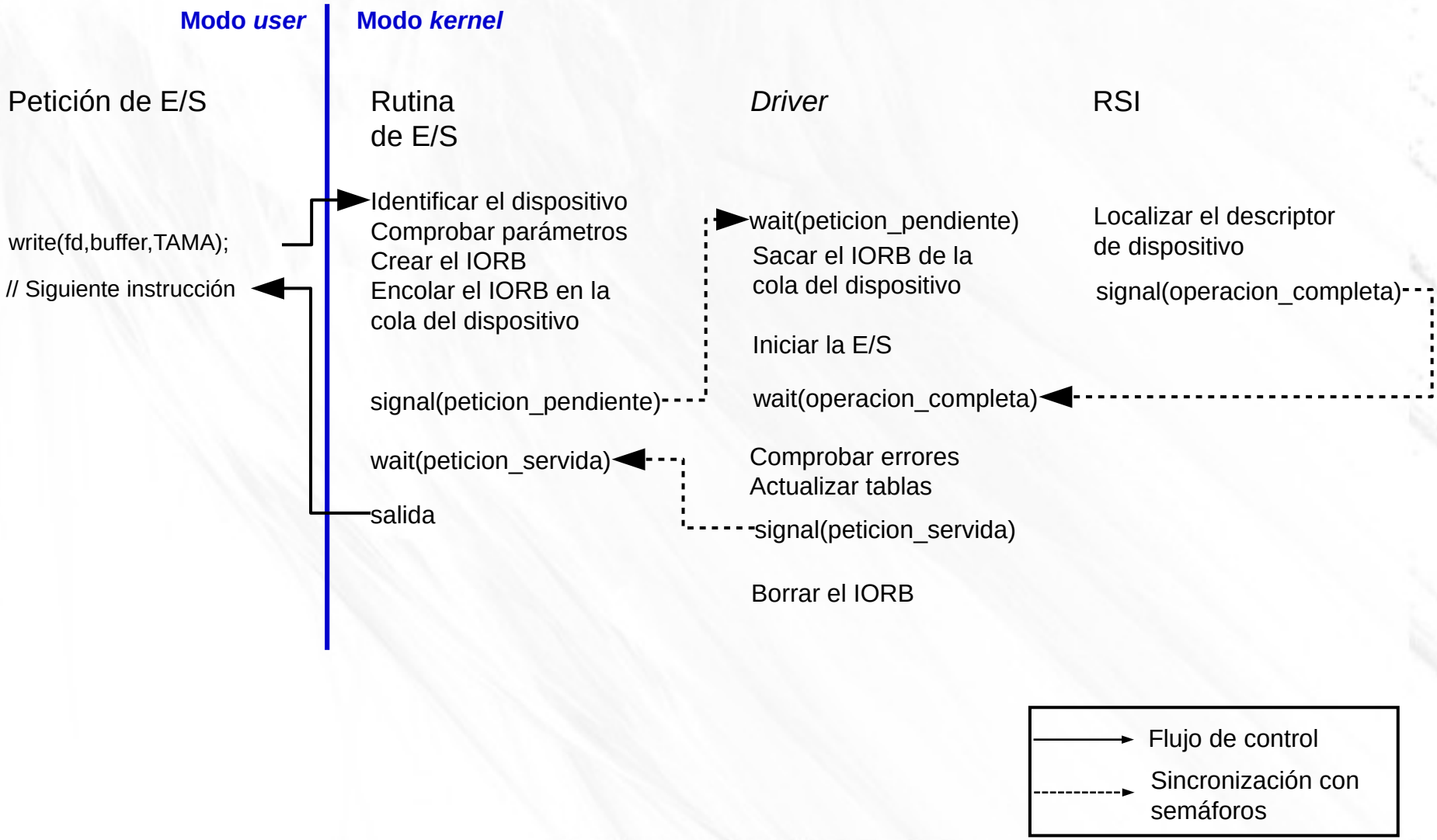
    `wait(peticion_servida)`

Controlador Software (*driver*)

- Contiene el **código dependiente de dispositivo** y conoce la estructura de registros del controlador hardware. Existe un *driver* por cada dispositivo (o tipo).
- Funcionamiento de un *driver* genérico:

```
While (true) {
  wait(peticion_pendiente);
  Sacar IORB de la cola de peticiones;
  Iniciar operación E/S.
  Esperar finalización: wait(operacion_completa);
  Comprobar errores;
  Notificar fin E/S al proceso solicitante:
  signal(peticion_servida);
  Eliminar IORB;
}
```

Funcionamiento del software de E/S



Contenidos

- Dispositivos de E/S
- Arquitectura software del sistema de E/S
- *Buffering*
- Planificación de disco
- Sistemas RAID

Buffering de E/S

- Restricciones de la E/S
- *Buffering* de E/S

Restricciones de la E/S

¿Cómo afecta la E/S al rendimiento del SO?

- Supongamos una lectura de varios bloques de disco al espacio virtual del proceso comenzando en la dirección 1000 mediante DMA:
  - El proceso debe permanecer en estado BLOQUEADO hasta que finalice la operación.
  - Los marcos de página (direcciones físicas) que alojan las direcciones virtuales implicadas deben mantenerse en RAM → Restringe las decisiones de gestión del conjunto residente e intercambio.
  - ¡Posibilidad de interbloqueo! Petición E/S → BLOQUEADO → intercambio → SUSP_BLOQUEADO → E/S espera por proceso en RAM

Buffering de E/S

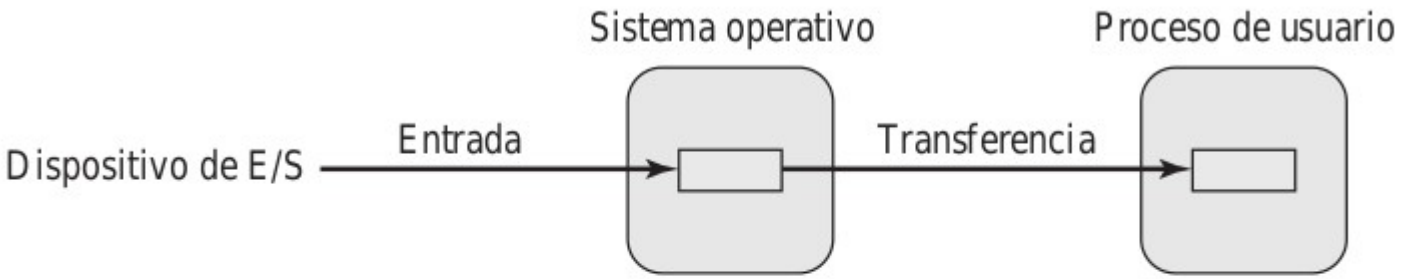
Solución

- ***Buffering* de E/S**. Realizar las transferencias de entrada antes de que se hagan las peticiones correspondientes y llevar a cabo las transferencias de salida un cierto tiempo después de que se haya hecho la petición.
- **Idea**. Cuando un proceso de usuario emite una petición de E/S, el sistema operativo asigna un *buffer* para la operación en memoria *kernel*:
  - Las lecturas de bloque usan el *buffer* y cuando se completa la transferencia se mueve la información a espacio de usuario y se solicita otro bloque.
  - Las escrituras usan el *buffer* y posteriormente serán realizadas las operaciones sobre disco.

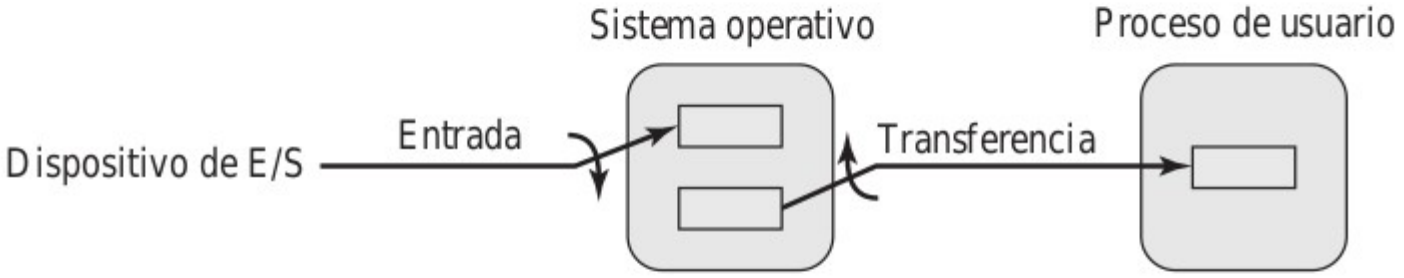
Variantes del *buffering* de E/S

- **Doble *buffer***. Se asignan dos *buffers* en memoria *kernel* a la operación de E/S. Un proceso transfiere datos a/desde un *buffer* mientas el SO vacía/llena el otro *buffer*.
- ***Buffer* circular**. Si un proceso solicita muchas peticiones de E/S el doble *buffer* puede ser insuficiente → utilizar varios *buffers*.

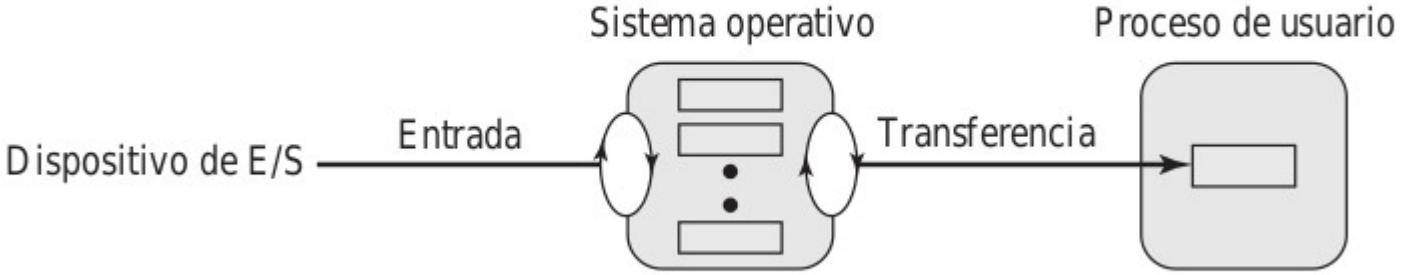
Variantes del *buffering* de E/S



(b) *Buffer* único



(c) *Buffer* doble



(d) *Buffer* circular From [Sta2005]

Contenidos

- Dispositivos de E/S
- Arquitectura software del sistema de E/S
- *Buffering*
- Planificación de disco
- Sistemas RAID

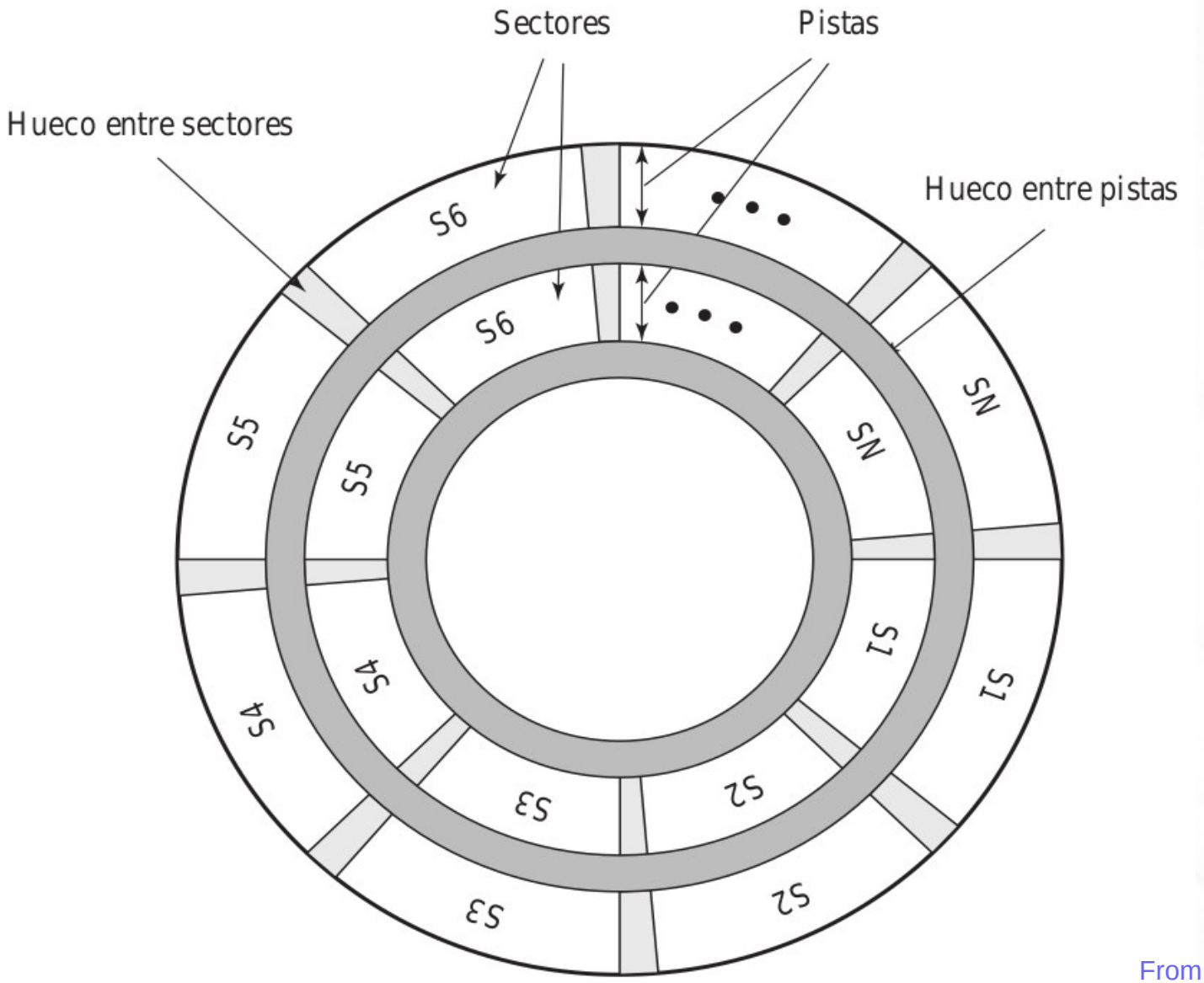
Planificación de disco

- Estructura de un disco magnético
- Planificación de disco

Estructura de un disco magnético

- Un disco es un plato circular construido de metal o plástico que está cubierto con un material magnético.
- Los datos se graban en el disco y se recuperan del mismo mediante una bobina conductora llamada **cabeza**.
- La cabeza permanece fija durante una operación de lectura o escritura mientras que el plato rota debajo de ella.
- La organización de los datos en el plato consiste en un conjunto concéntrico de anillos (**pistas**) y cada pista tiene la misma anchura que la cabeza.
- Los datos se transfieren del disco en unidades de tamaño fijo denominadas **sectores**.

Estructura de un disco magnético

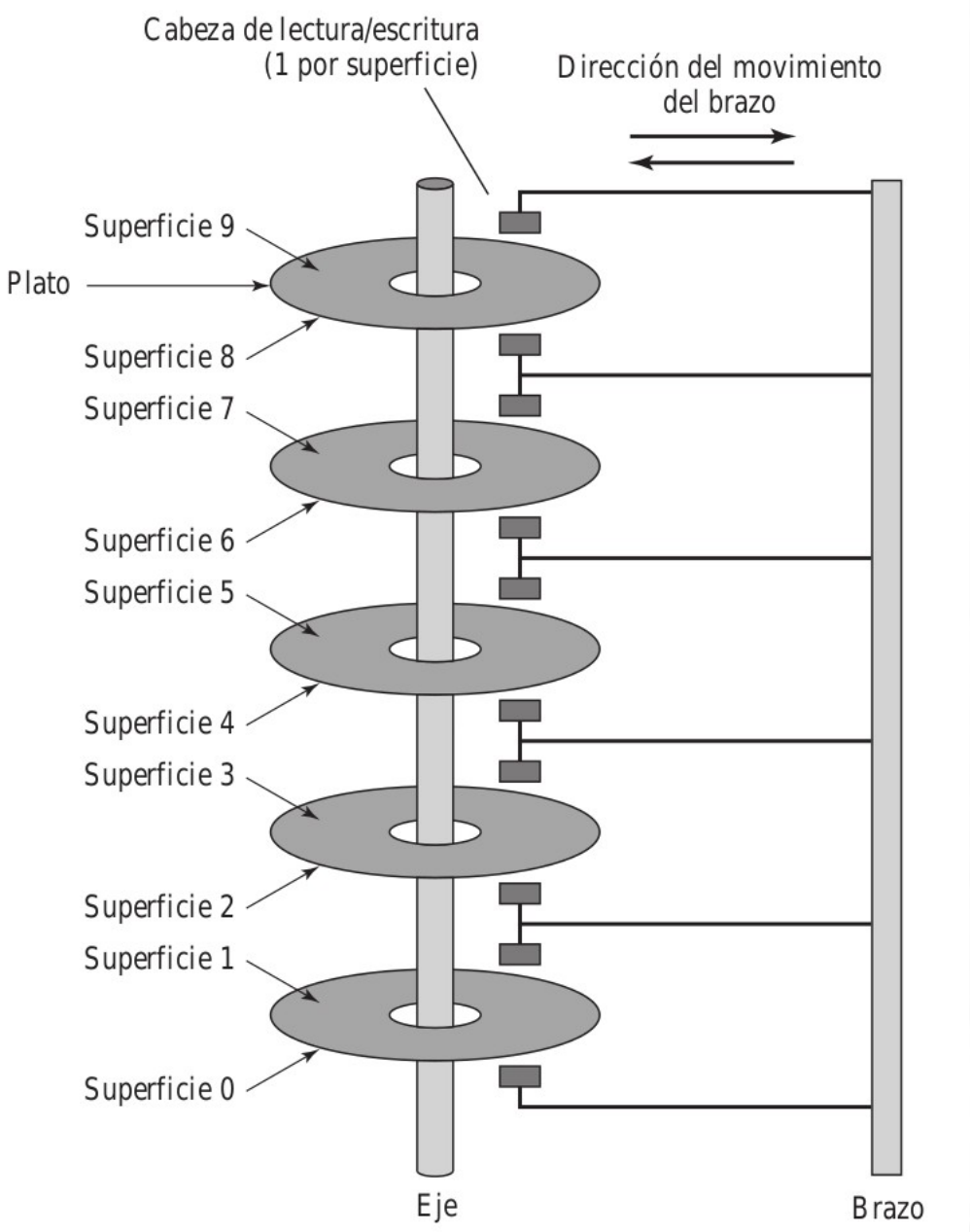


From [Sta2005]

Estructura de un disco magnético

- **Disco con cabeza fija**. Hay una cabeza de lectura/escritura por pista. Todas las cabezas están montadas en un brazo rígido que se extiende a lo largo de la superficie.
- **Disco de cabeza móvil**. Hay sólo una cabeza de lectura/escritura por cada superficie. También en este caso la cabeza está montada en un brazo pero el brazo puede moverse radialmente para situar la cabeza sobre cualquier pista de la superficie.
- Los discos duros incluyen múltiples platos apilados verticalmente con una cabeza móvil por superficie. Todas las cabezas se posicionan conjuntamente sobre pistas que están a la misma distancia del centro del disco (**cilindro**).

Estructura de un disco magnético



From [Sta2005]

Estructura de disco para el SO

- El SO ve el disco como una secuencia de bloques (B_0, B_1, B_2,..., B_{n-1}) de tamaño n (bloques lógicos).
- La información física de disco se referencia por una dirección compuesta:

  `(dispositivo,superficie,pista,sector)`

- Existe una correspondencia entre dirección de bloque lógico, B_i, y dirección física (dispositivo, superficie, pista, sector).
- Un bloque lógico puede estar formado por 1 o potencia de 2 sectores.

Planificación de disco

- Una petición de bloque de disco consta de tres fases:
  - **Tiempo de búsqueda**. Posicionamiento de la cabeza en el cilindro adecuado.
  - **Latencia rotacional**. Tiempo de espera hasta que pase el sector referenciado bajo la cabeza.
  - **Transferencia de datos** a/desde RAM.
- Cuando un proceso emite una petición de E/S, su IORB debe esperar en la cola del dispositivo a ser atendido.
- El SO puede mejorar el tiempo medio de servicio de disco planificando la cola de IORBs.
- La planificación intenta **minimizar el tiempo de búsqueda**, es decir la distancia de posicionamiento de la cabeza de lectura/escritura.

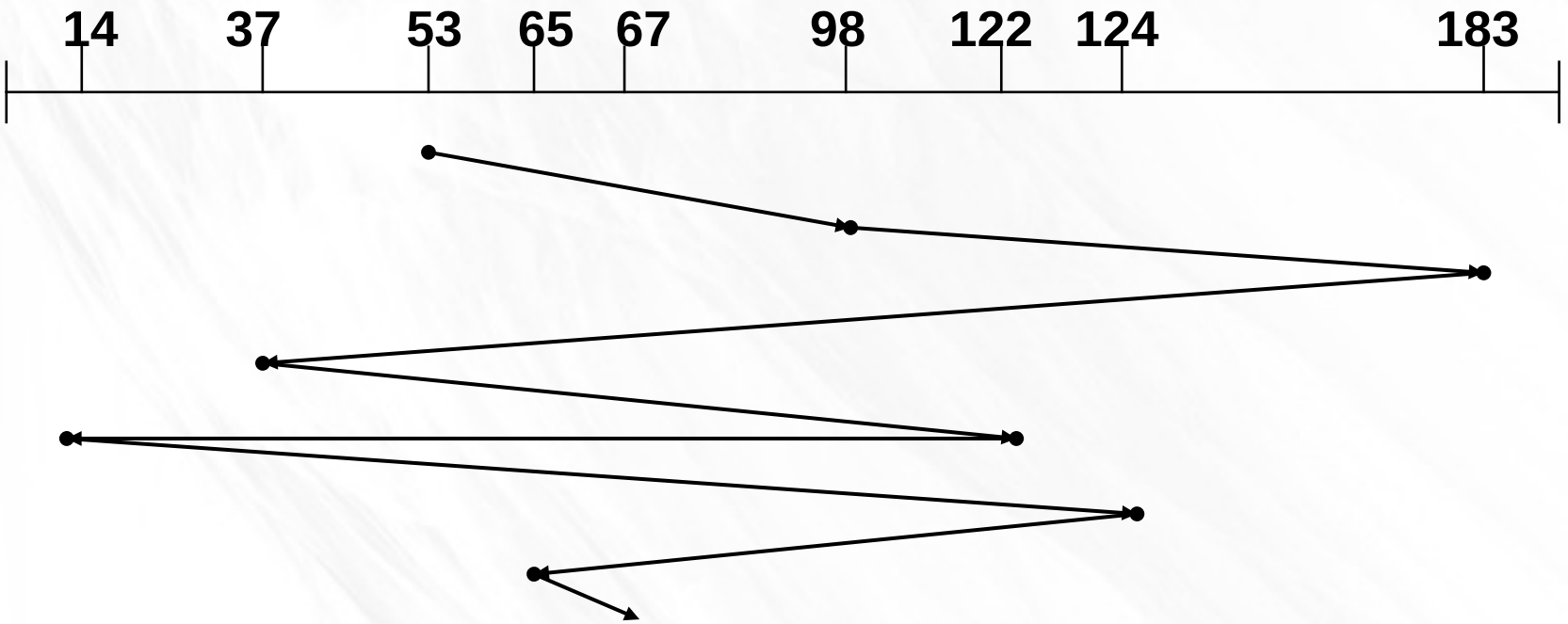
Planificación de disco

- Algunos algoritmos de planificación de disco: FCFS, SSTF, SCAN, C-SCAN, LOOK, **C-LOOK**.
- Veamos un ejemplo para comprobar el comportamiento de los algoritmos:

  Cola de peticiones (número de pista): (98, 183, 37, 122, 14, 124, 65, 67). Inicialmente la cabeza está posicionada en la pista 53.

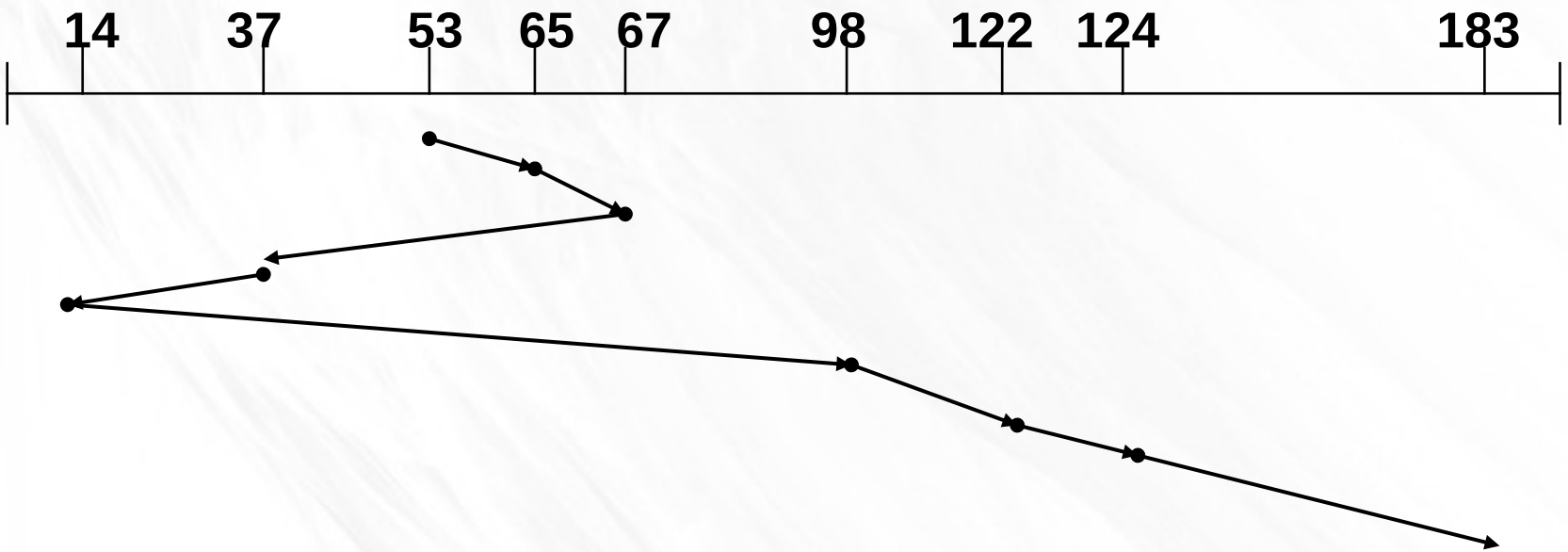
Planificación de disco

- FCFS (*First Come First Served*).



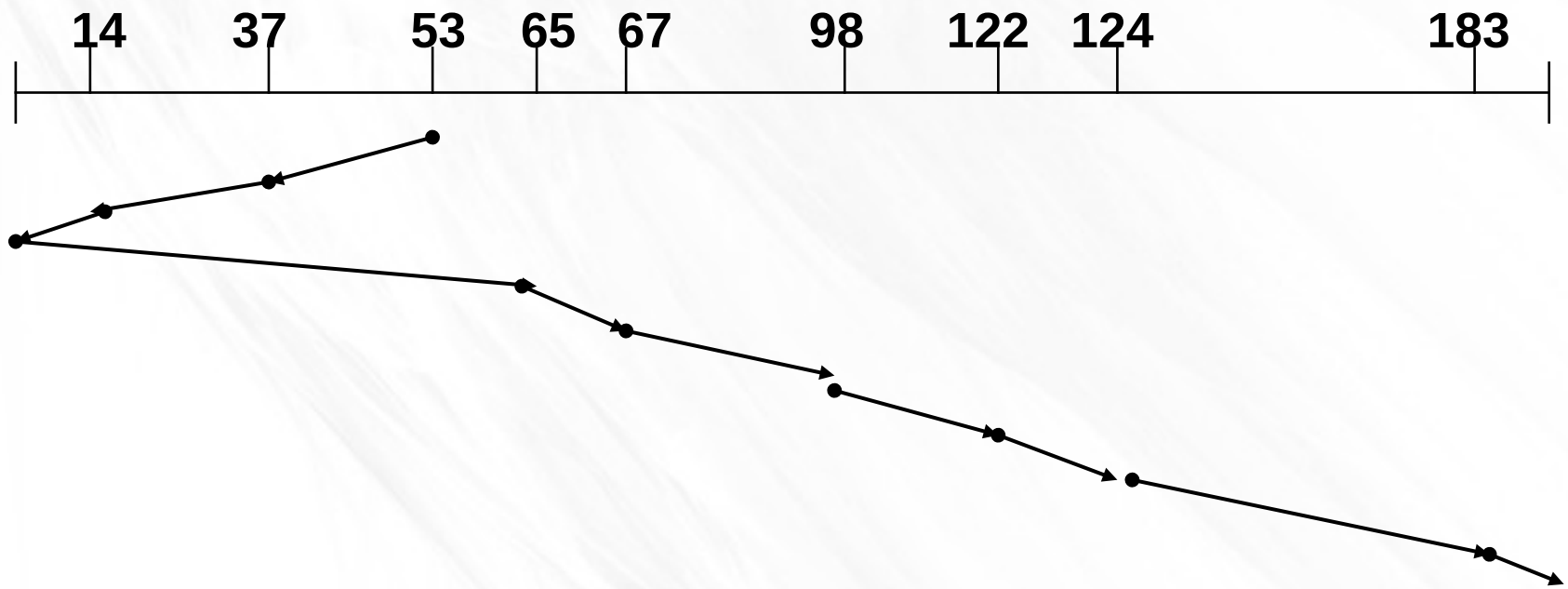
Planificación de disco

- SSTF (*Shortest Seek Time First*). Primero la petición con menor distancia de posicionamiento de la cabeza.



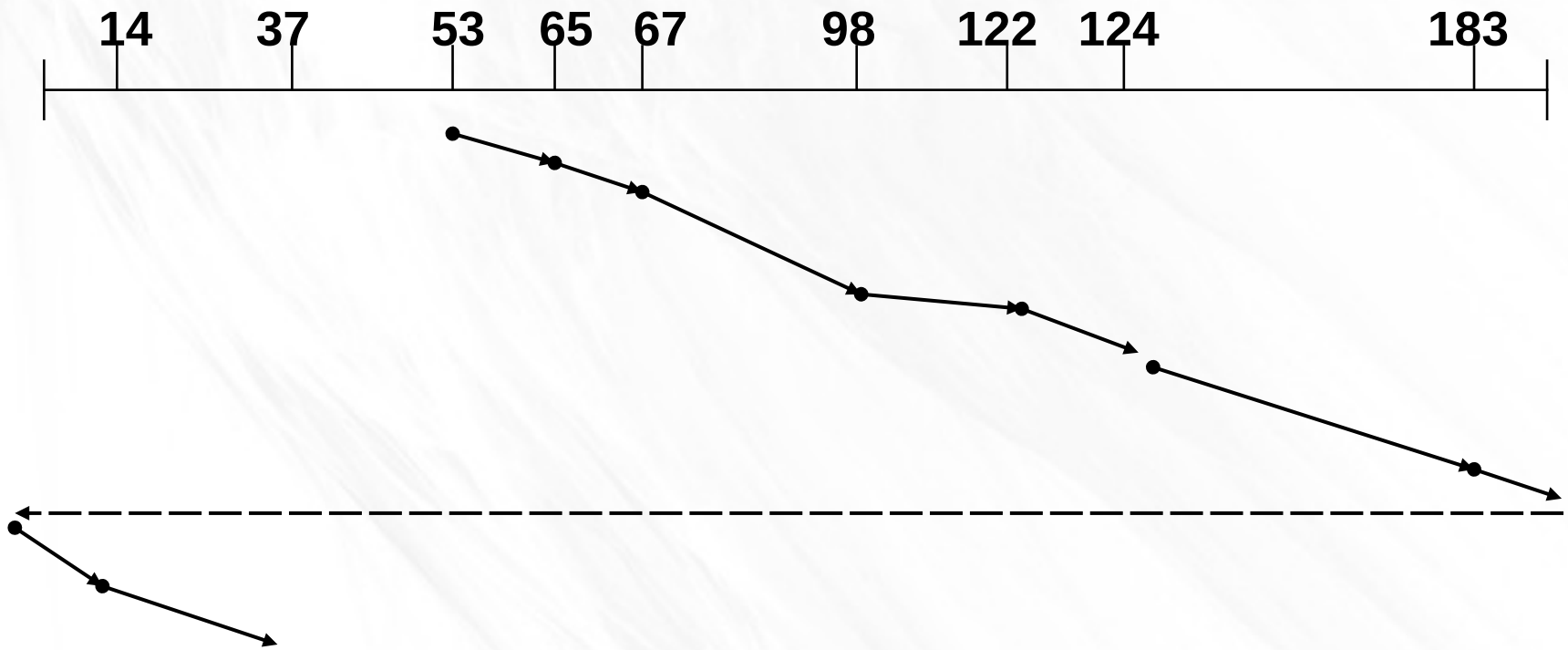
Planificación de disco

- SCAN. Recorrido de todas las pistas de primera pista a última pista y viceversa.



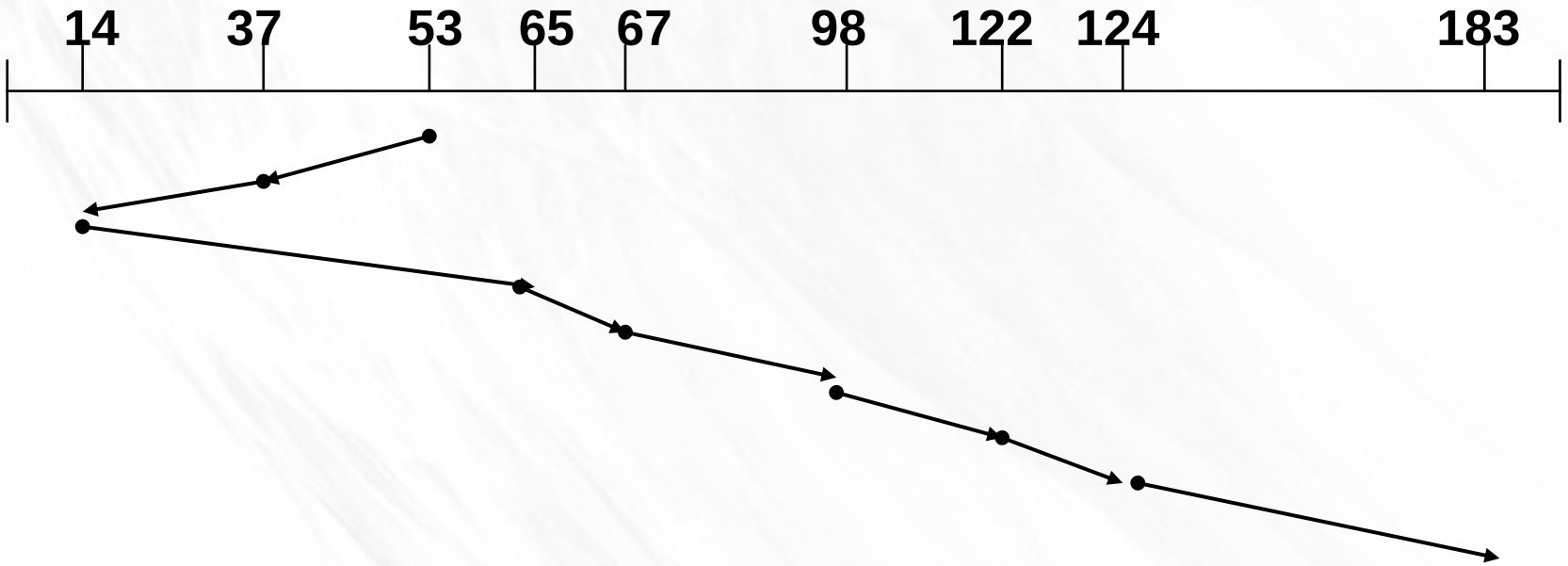
Planificación de disco

- C-SCAN. Recorrido de todas las pistas de primera pista a última pista y vuelta a la primera pista.



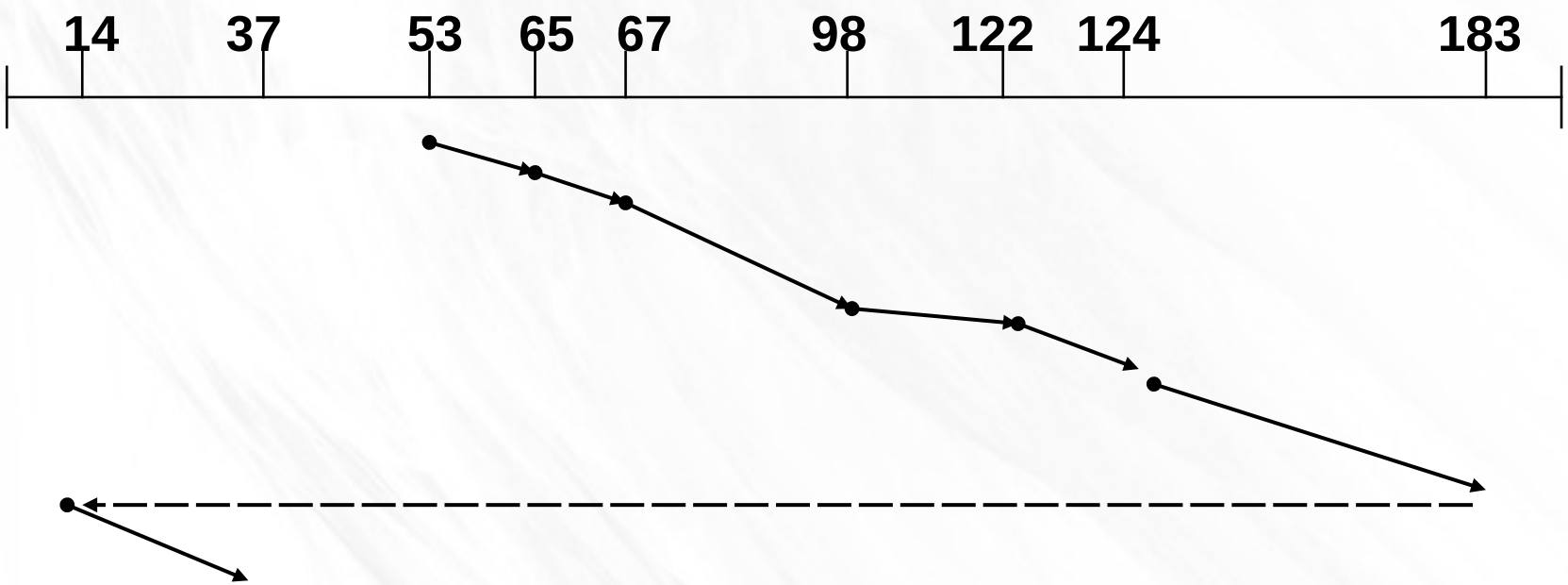
Planificación de disco

- LOOK. Optimización del SCAN. La cabeza solamente se mueve en una dirección si quedan peticiones.



Planificación de disco

- **C-LOOK**. Optimización de C-SCAN.



Contenidos

- Dispositivos de E/S
- Arquitectura software del sistema de E/S
- *Buffering*
- Planificación de disco
- Sistemas RAID

Sistemas RAID

- Sistemas RAID
- Niveles RAID

Sistemas RAID

- **Primera idea**. Si un componente del ordenador es difícil de mejorar desde el punto de vista tecnológico, entonces utilizar múltiples componentes en paralelo.
- Un vector de discos (*disk array*) permite:
  - Manejar en paralelo las peticiones de E/S independientes si los bloques están en discos distintos.
  - Una única petición de E/S se puede ejecutar en paralelo si el bloque de datos que se pretende acceder está distribuido entre los discos.
- Por tanto, permiten mejorar el **rendimiento de la E/S**.

Sistemas RAID

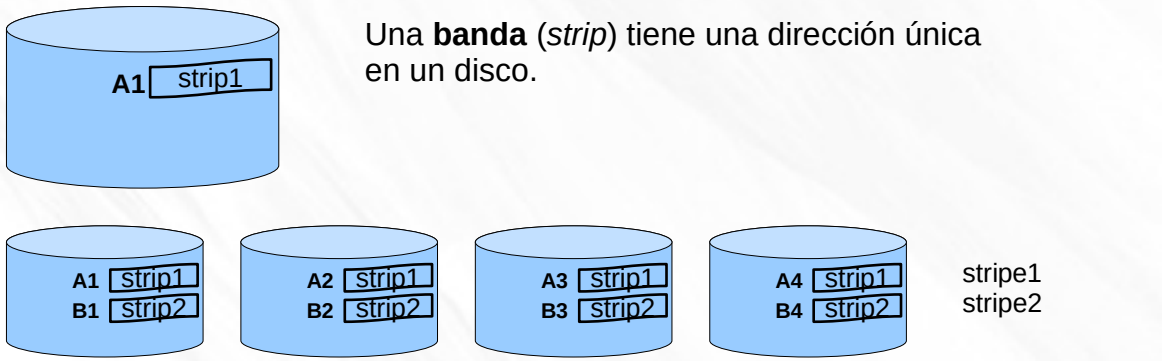
- **Segunda idea**. Los vectores de disco facilitan la protección de datos.
- **Algunos inconvenientes de los discos**:
  - Los discos se rompen.
  - Los discos pueden tener sectores defectuosos.
- Estos inconvenientes provocan perdida de datos, lo que va en contra de la **tolerancia a fallos**.
- Para prevenir la pérdida de datos podemos:
  - Tener una copia de los datos (*mirroring*).
  - Tener un mecanismo que permita reconstruir los datos perdidos (paridad).

Sistemas RAID

- **RAID (*Redundant Array of Independent Disks*)**. Estándar de almacenamiento de datos que utiliza un vector de discos (HDD o SSD) entre los cuales se distribuyen o se replican los datos.
- El estándar RAID consta de siete niveles (0 al 6) que determinan distintas configuraciones que tienen en común:
  - Conjunto de unidades físicas de disco tratado por el SO como un único **dispositivo lógico**.
  - Los datos están distribuidos entre las distintas unidades físicas (***striping* de disco**).
  - La capacidad de redundancia del conjunto se utiliza para almacenar **información de paridad** → los datos se pueden recuperar si falla un disco.

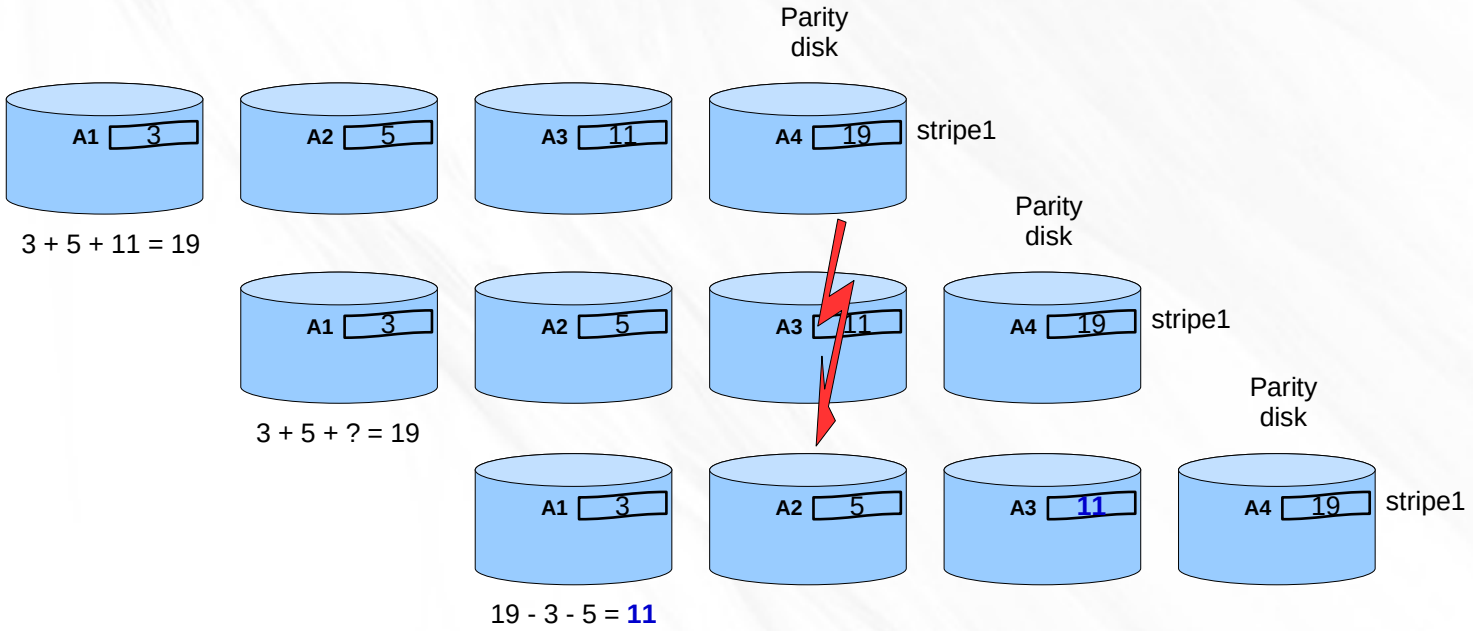
Striping de disco

- El dispositivo lógico está dividido en bandas (bloques físicos, sectores u otra unidad).
- Las bandas (*strip*) se asignan secuencialmente a discos consecutivos del vector.
- Una lista (*stripe*) es una secuencia consecutiva de bandas, cada una en un disco.



Paridad

- Paridad es un esquema de protección frente a errores que proporciona tolerancia a fallos en un conjunto de datos cuando se “pierde” alguno.
- El más simple es XOR (suma).
- Sumar (XOR) los valores de cada *strip* y sumar (XOR) los valores resultantes para calcular la paridad.



Beneficios de los sistemas RAID

- Tolerancia a fallos (*fault tolerance*):
  - Los niveles RAID (menos RAID 0) brindan protección contra errores irrecuperables de lectura de sector, así como contra fallos de unidades disco.
  - Los niveles a partir de RAID 2 emplean un esquema de protección contra errores (paridad).
- Rendimiento (*performance*):
  - Transferencia de datos (*data throughput*).
  - Tasa de peticiones de E/S (*I/O request rate*), es decir, capacidad de satisfacer peticiones de E/S.
- Capacidad.

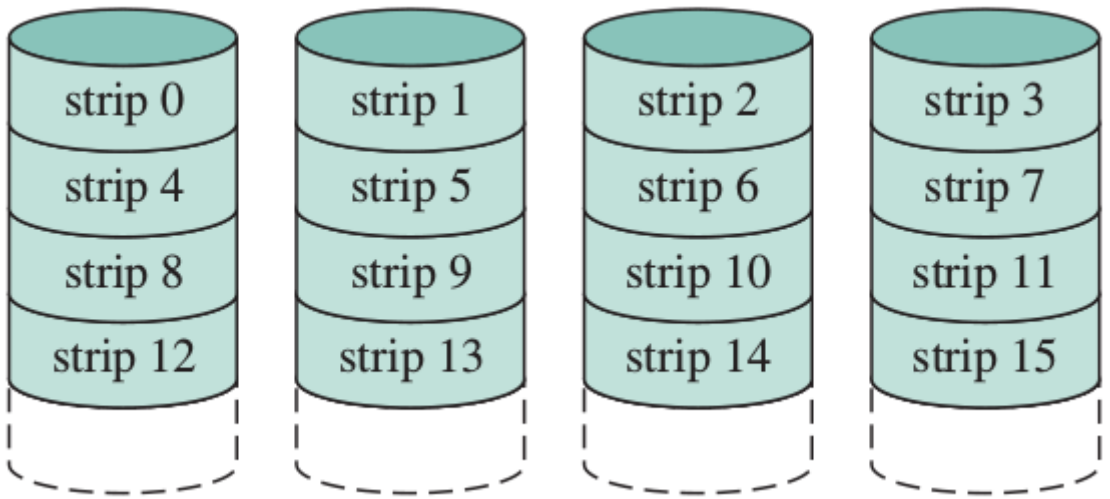
Rendimiento de la E/S en RAID

- Vamos a fijarnos en tres aspectos para medir el rendimiento de la E/S en los distintos niveles RAID:
  - Capacidad de transferencia de datos.
  - Productividad: Número de peticiones de E/S satisfechas por unidad de tiempo (*I/O request rate*). Para la productividad distinguimos si la petición es de lectura o de escritura.
  - Tolerancia a fallos en el sentido de disponibilidad de los datos (*data availability*)
- Supondremos un vector de N discos de datos para evaluar los diferentes niveles RAID.

RAID 0 (Listas sin paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 sector u otra unidad (varios sectores).
- Este nivel no implementa ni *mirroring* ni paridad.
- Si una única petición de E/S consta de múltiples bandas contiguas lógicamente, se pueden manejar en paralelo hasta N bandas de esta petición con lo que aumenta la capacidad de transferencia de datos.
- No se pueden servir múltiples peticiones de E/S simultáneamente por lo que no aumenta la productividad.
- Si falla un disco del vector se pierde toda la información.

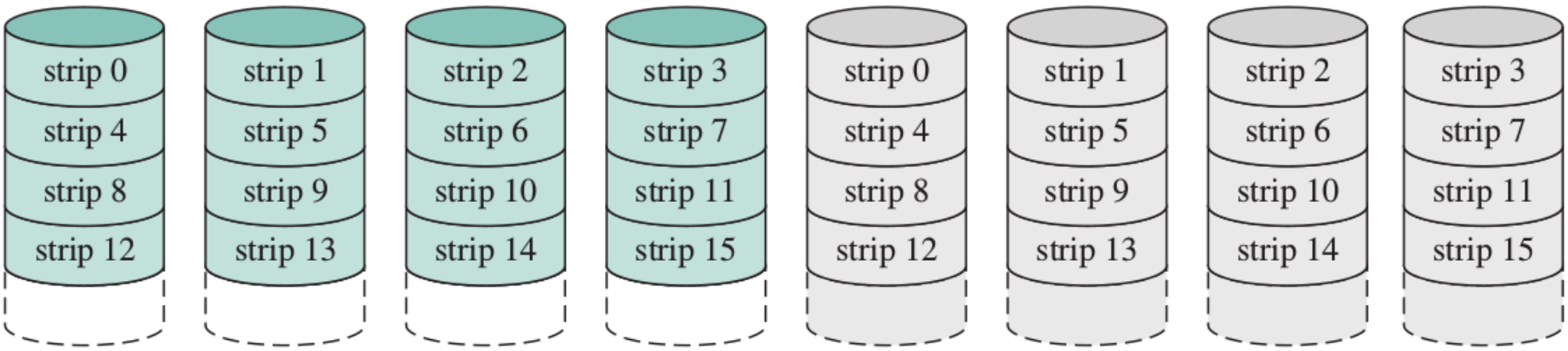
RAID 0 (Listas sin paridad)



RAID 1 (Listas con *mirroring* sin paridad)

- El más simple solamente consiste en dos discos replicados. Una lista compuesta de una sola banda que identifica un bloque de disco.
- En la implementación más interesante las listas están compuestas por bandas que en este caso identifican 1 bloque y cada banda está replicada en el vector.
- Incrementa la capacidad de transferencia de datos de RAID0 al beneficiarse de la replicación de las bandas.
- Aumenta la productividad debido a poder servir simultáneamente peticiones de lectura desde las réplicas.
- Las escrituras deben realizarse en todas las réplicas por lo que se ven penalizadas.
- Si falla una réplica se pueden seguir usando las otras.

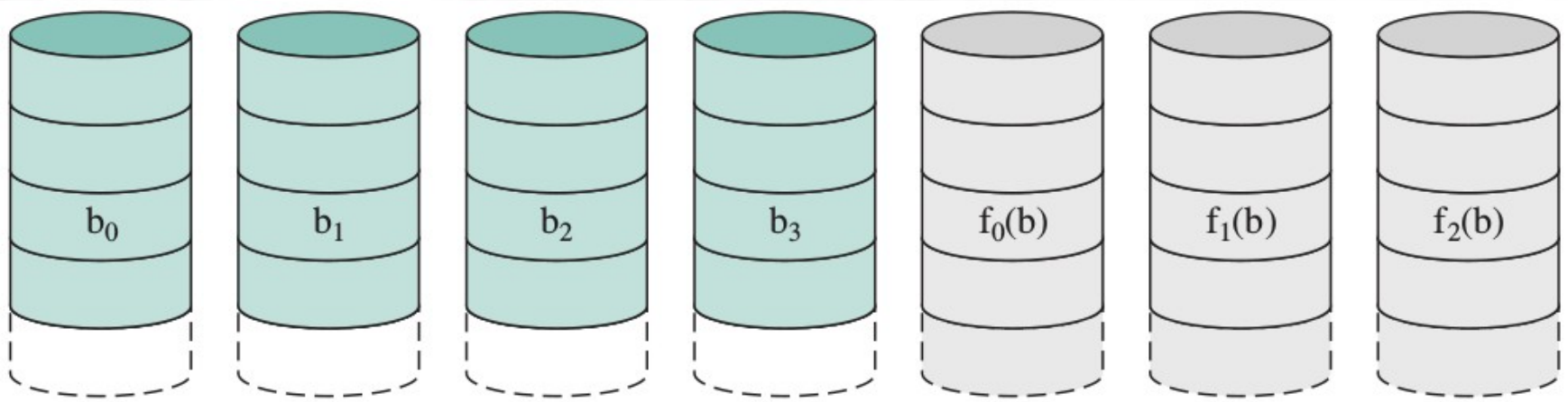
RAID 1 (Listas con *mirroring* sin paridad)



RAID 2 (Acceso paralelo y paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 byte o una palabra.
- Se calcula un código de corrección de error (código Hamming) con los bits correspondientes de cada disco del vector y los bit resultantes se almacenan en el/los disco(s) de paridad. El vector requiere N+m discos y m depende de la paridad.
- Mantiene la capacidad de transferencia de datos de RAID0 al usar bandas.
- No se pueden servir múltiples peticiones de E/S simultáneamente por lo que no aumenta la productividad.
- Escrituras penalizadas por el cálculo del código Hamming y la escritura en el/los disco(s) de paridad.

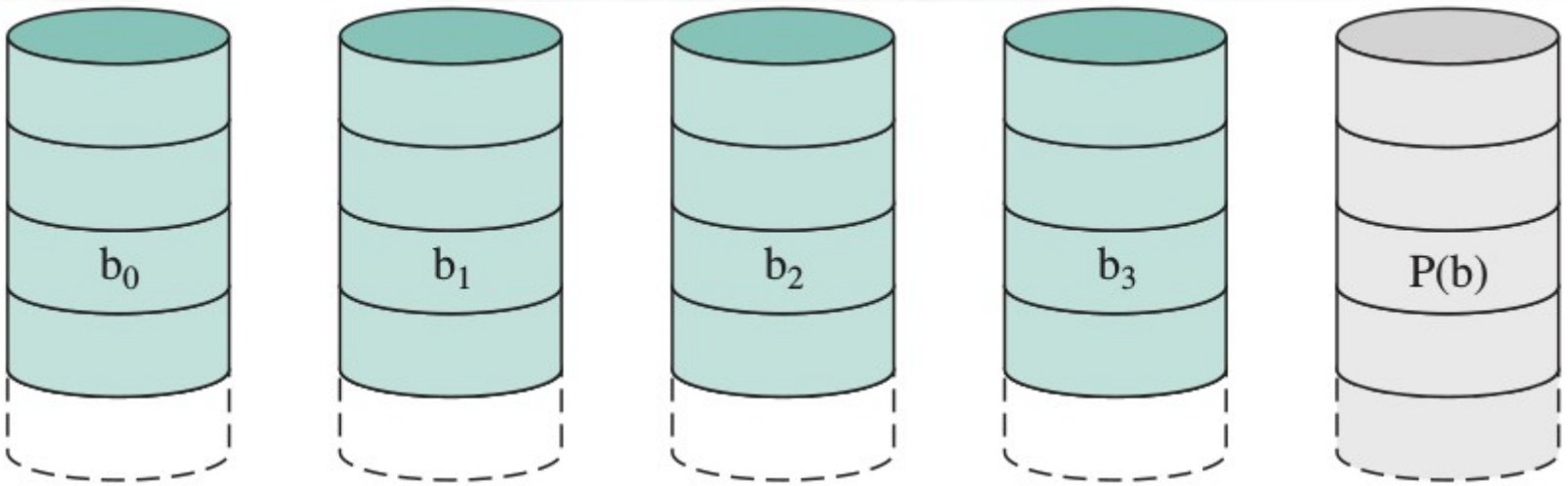
RAID 2 (Acceso paralelo y paridad)



RAID 3 (Acceso paralelo y paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 byte o una palabra.
- Calcula un bit de paridad simple para todos los bits almacenados en la misma posición de cada banda de la lista. El vector requiere N+1 discos (un solo disco de paridad).
- Mantiene la capacidad de transferencia de datos de RAID0 al usar bandas.
- No se pueden servir múltiples peticiones de E/S simultáneamente por lo que no aumenta la productividad.
- Escrituras penalizadas (menos que RAID2) por el cálculo del bit de paridad y la escritura en el único disco de paridad.

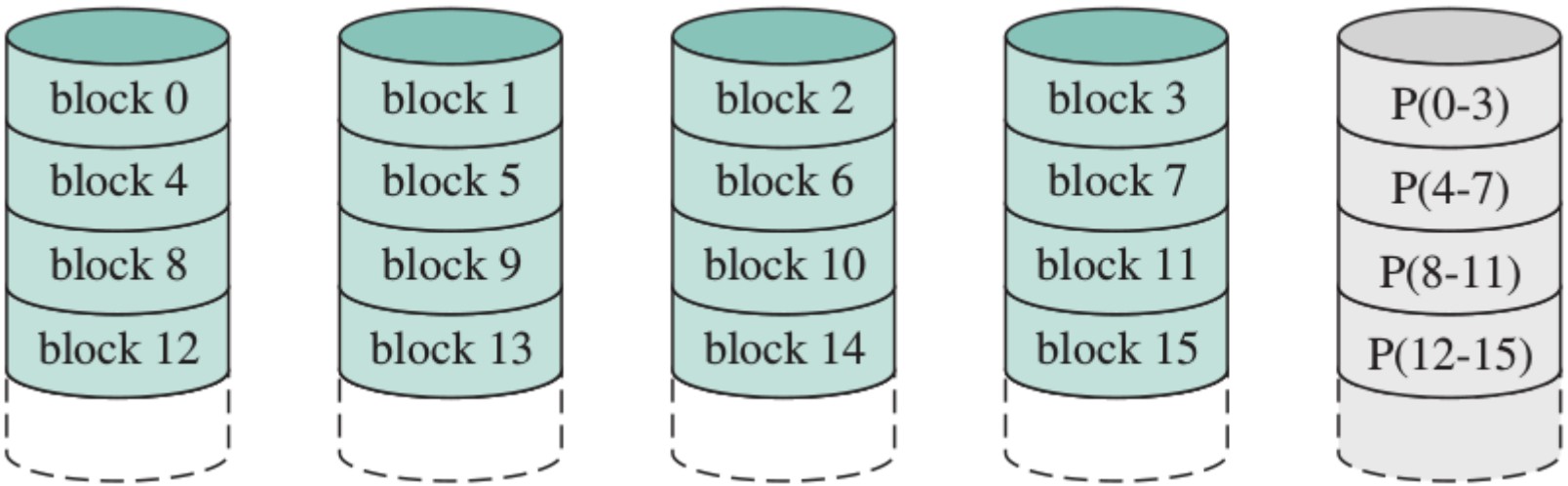
RAID 3 (Acceso paralelo y paridad)



RAID 4 (Acceso independiente y paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 bloque de disco.
- Se calcula bit a bit una banda de paridad a partir de las bandas de la misma lista, almacenándose los bits de paridad resultantes en la banda correspondiente del disco de paridad. El vector requiere N+1 discos (un solo disco de paridad).
- La capacidad de transferencia de datos es equivalente a un solo disco por ser acceso independiente.
- Se pueden servir múltiples peticiones de E/S simultáneamente por lo que aumenta la productividad.
- Escrituras penalizadas por el cálculo de la paridad de la lista correspondiente y la escritura en el disco de paridad de la banda (cuello de botella).

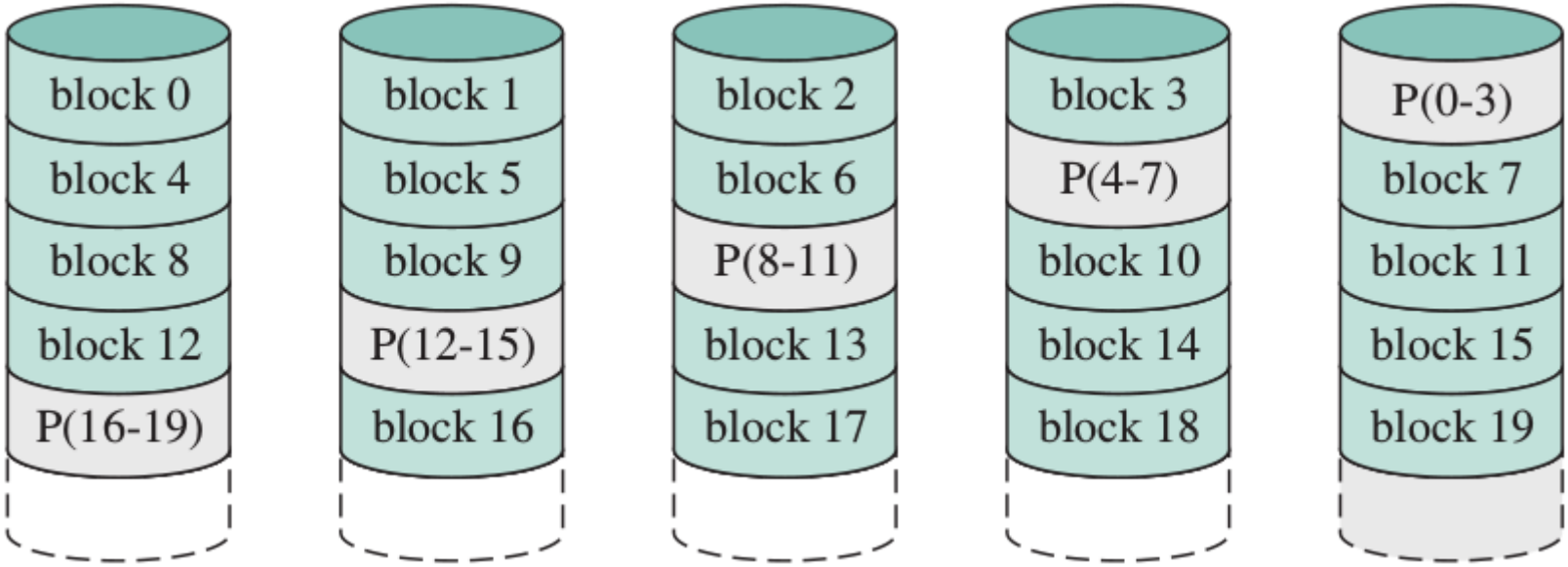
RAID 4 (Acceso independiente y paridad)



RAID 5 (Acceso independiente y paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 bloque de disco.
- Se calcula bit a bit una banda de paridad a partir de las bandas de la misma lista, almacenándose los bits de paridad resultantes en la banda correspondiente que es distribuida en todos los discos del vector. El vector requiere N+1 discos (1 extra por la paridad).
- La capacidad de transferencia de datos es equivalente a un solo disco por ser acceso independiente.
- Se pueden servir múltiples peticiones de E/S simultáneamente por lo que aumenta la productividad.
- Escrituras penalizadas por el cálculo de la paridad de la lista correspondiente y la escritura de la banda de paridad en su disco correspondiente.

RAID 5 (Acceso independiente y paridad)



RAID 6 (Acceso independiente y paridad)

- Las listas están compuestas por bandas que en este caso identifican 1 bloque de disco.
- Se calcula bit a bit una banda de paridad a partir de las bandas de la misma lista, almacenándose los bits de paridad resultantes en la banda correspondiente que es distribuida en todos los discos del vector. El vector requiere N+2 discos (2 discos extra).
- La capacidad de transferencia de datos es equivalente a un solo disco por ser acceso independiente.
- Se pueden servir múltiples peticiones de E/S simultáneamente por lo que aumenta la productividad.
- Escrituras doblemente penalizadas con respecto a RAID5 por el doble cálculo de paridad y la doble escritura de las bandas de paridad.

RAID 6 (Acceso independiente y paridad)

